LCP A — Exam Study Sheet

5 versions × (3–5 exercises + 1 bash) = 23 exercises total

1 Common Imports (all versions)

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit, minimize
from scipy import stats, integrate, fftpack
from scipy.special import gamma
from scipy.interpolate import UnivariateSpline
import pandas as pd
```

2 Version 1

2.1 Ex.9 – Populations (hares, lynxes, carrots)

RECIPE

1. Load with `np.loadtxt`, unpack with `.T`
2. Plot 3 lines, compute mean/std, correlation matrix
3. `argmax` along correct axis for highest each year

```
data = np.loadtxt('populations.txt')
year, hares, lynxes, carrots = data.T

plt.plot(year, hares, 'b-o', label='Hares')
plt.plot(year, lynxes, 'r-s', label='Lynxes')
plt.plot(year, carrots, 'g-^', label='Carrots')

populations = np.array([hares, lynxes, carrots])
print("Mean:", hares.mean(), lynxes.mean())
print("Std:", hares.std(), lynxes.std())
print("Correlation:\n", np.corrcoef(populations))

species = ['Hares', 'Lynxes', 'Carrots']
highest_idx = np.argmax(populations, axis=0)
for i, y in enumerate(year):
    print(f"{int(y)}: {species[highest_idx[i]]}")
```

Key: `populations` is (3×21), so `axis=0` picks max across 3 species for each year.

2.2 Ex.2 – Alaska Temperature Curve Fitting

RECIPE

1. Define sinusoidal model: $T(x) = A \sin\left(\frac{2\pi(x-C)}{12}\right) + D$
2. Use `curve_fit` with good initial guesses for max and min
3. Plot data + fit

```
months = np.arange(1, 13)
max_temp = np.array
    ([17,19,21,28,33,38,37,37,31,23,19,18])
min_temp = np.array([-62,-59,-56,-46,-32,-18,
                     -9,-13,-25,-46,-52,-58])

def temp_model(x, A, C, D):
    return A * np.sin(2*np.pi*(x - C)/12) + D

# INITIAL GUESSES     important to get right!
p0_max = [10, 4, 27]   # small amp, phase~4, mean~27
p0_min = [25, 4, -35]  # big amp, same phase, mean~-35

popt_max, _ = curve_fit(temp_model, months,
                        max_temp, p0=p0_max)
popt_min, _ = curve_fit(temp_model, months,
                        min_temp, p0=p0_min)

x_fine = np.linspace(1, 12, 100)
plt.plot(x_fine, temp_model(x_fine, *popt_max))
plt.plot(x_fine, temp_model(x_fine, *popt_min))
```

Result: Phase C similar for both $\Rightarrow$ peak at same time (summer).

2.3 Ex.1 – Radioactive Decay (^{208}Tl $\rightarrow$ ^{208}Pb)

KEY FORMULAS

- Decay probability per Δt: $p = 1 - 2^{-\Delta t/\tau}$
- PDF: $p(t) = \frac{\ln 2}{\tau} 2^{-t/\tau}$
- Inverse CDF: $t = -\tau \log_2(r), \quad r \sim \mathcal{U}(0,1)$

2.3.1 Method 1: Step-by-step MC

```
N0 = 1000; tau = 3.052*60; dt = 1  # seconds
times = np.arange(0, 1500, dt)
Tl = np.zeros(len(times)); Tl[0] = N0
Pb = np.zeros(len(times))

p_decay = 1 - 2**(-dt/tau)
for i in range(1, len(times)):
    n_decayed = np.sum(
        np.random.random(int(Tl[i-1])) < p_decay)
    Tl[i] = Tl[i-1] - n_decayed
    Pb[i] = Pb[i-1] + n_decayed
```

2.3.2 Method 2: Inverse transform

```
r = np.random.random(N0)
decay_times = -tau * np.log2(r)
decay_times = np.sort(decay_times)

times2 = np.linspace(0, 1500, 500)
Tl2 = np.array([N0 - np.sum(decay_times < t)
                for t in times2])
# Compare with theory:
plt.plot(times2, N0 * 2**(-times2/tau), 'r--')
```

2.4 Ex.1 – Kernel Density Estimation

RECIPE

1. Generate data: `np.random.normal(5, 2, 100)`
2. Histogram with Poisson errors $\sqrt{N_{\text{bin}}}$
3. KDE: Gaussian at each data point, bandwidth $h = 1.06\,\hat{\sigma}\,N^{-1/5}$
4. Sum Gaussians, normalize to match histogram integral

```
N = 100; x = np.random.normal(5, 2, N)

# Histogram
counts, bin_edges, _ = plt.hist(x, bins=15,
                                alpha=0.5)
bin_centers = (bin_edges[:-1] + bin_edges[1:]) / 2
bin_width = bin_edges[1] - bin_edges[0]
errors = np.sqrt(counts)
plt.errorbar(bin_centers, counts, yerr=errors,
             fmt='ko', capsize=3)

# KDE
h = 1.06 * x.std() * N**(-1/5)  # Scott's rule
x_range = np.linspace(x.min()-3*h, x.max()+3*h, 200)

kde_sum = np.zeros_like(x_range)
for xi in x:
    kde_sum += stats.norm(loc=xi, scale=h).pdf(x_range)

# Normalize: match histogram area
hist_integral = counts.sum() * bin_width
kde_integral = integrate.trapezoid(kde_sum, x_range)
kde_normalized = kde_sum * hist_integral / kde_integral
plt.plot(x_range, kde_normalized, 'r-', lw=2)
```

2.5 Bash 1 – Student File Management

WARNING: BASH Ex 1

Don't forget the Shebang! Always verify that the very first line of your script is exactly `#!/bin/bash` before writing any other commands.

```
# 1.a: Create dir + conditional download
cd $HOME; mkdir -p students; cd students
if [ ! -f "LCP_22-23_students.csv" ]; then
    wget https://www.dropbox.com/.../LCP_22-23_students.
        csv
```

```
fi

# 1.b: Filter by program
grep "PoD" LCP_22-23_students.csv > pod.csv
grep "Physics" LCP_22-23_students.csv > physics.csv

# 1.c: Count by initial letter
for letter in {A..Z}; do
    count=$(grep -c "^$letter" LCP_22-23_students.csv)
    echo "$letter: $count"
done

# 1.d: Find max letter
max_letter=""; max_count=0
for letter in {A..Z}; do
    count=$(grep -c "^$letter" LCP_22-23_students.csv)
    if [ $count -gt $max_count ]; then
        max_count=$count; max_letter=$letter
    fi
done
echo "Most common: $max_letter ($max_count)"

# 1.e: Group modulo 18
for i in {1..18}; do
    awk "NR % 18 == $i % 18" \
        LCP_22-23_students.csv > group_$i.csv
done
```

BASH CHEAT SHEET

`grep "pattern" file` = search lines
`grep -c` = count matches | `grep -v` = invert
`^` = start-of-line | `>` = redirect output
`-f` = test file exists | `-gt` = greater than
`$(cmd)` = command substitution
`awk 'cond{action}'` = process text line-by-line
`NR` = line number | `$1,$2..` = fields
`sed 's/old/new/g'` = find & replace
`wc -l` = count lines | `|` = pipe

3 Version 2

3.1 Ex.3 – Six-Hump Camelback Optimization

RECIPE

1. Define function $f(x, y)$
2. Visualize with `meshgrid` + `imshow`
3. `minimize(f, x0, method='BFGS')` from 5 starting points
4. Find global min: smallest f value among results

```
def camelback(xy):
    x, y = xy
    return (4-2.1*x**2+x**4/3)*x**2 + x*y \
           + (4*y**2-4)*y**2

# Visualize
x = np.linspace(-2, 2, 200)
y = np.linspace(-1, 1, 200)
X, Y = np.meshgrid(x, y)
Z = camelback([X, Y])
plt.imshow(Z, extent=[-2,2,-1,1],
           origin='lower', cmap='viridis')

# Minimize from multiple starts
starts = [(-1,0.5),(1,-0.5),(-1,-0.5),
          (1,0.5),(0,0)]
results = []
for p0 in starts:
    res = minimize(camelback, p0, method='BFGS')
    results.append((res.x, res.fun))

min_val = min(r[1] for r in results)
global_min = [r for r in results
              if np.isclose(r[1], min_val, atol=1e-5)]
```

Answer: 2 global minima at $(\pm 0.0898, \mp 0.7126)$, $f_{\min} \approx -1.0316$. Starting at $(0, 0) \rightarrow$ **local** min!

3.2 Ex.4 – Linspace and Slicing

```
x = np.linspace(0, 2*np.pi, 100)
every_10th = x[::10]        # every 10th element
reversed_x = x[::-1]        # reversed

# Where |sin(x) - cos(x)| < 0.1
mask = np.abs(np.sin(x) - np.cos(x)) < 0.1
x_close = x[mask]  # near pi/4 and 5*pi/4

plt.plot(x, np.sin(x), 'b-', label='sin')
plt.plot(x, np.cos(x), 'r-', label='cos')
plt.scatter(x_close, np.sin(x_close), c='g', s=100)
```

3.3 Ex.1 – PCA on 3D Dataset

KEY FORMULAS

- Center: $\mathbf{X}_c = \mathbf{X} - \bar{\mathbf{X}}$
- Covariance: $\mathbf{C} = \text{cov}(\mathbf{X}_c^T)$
- Eigen: $\lambda, \mathbf{v}$ from $\mathbf{C}$, sorted by $\lambda \downarrow$
- SVD: $\lambda_i = \sigma_i^2/(N-1)$
- Variance explained: $\lambda_i / \sum \lambda_j$

```
N = 1000
x1 = np.random.normal(0, 1, N)
x2 = x1 + np.random.normal(0, 3, N)
x3 = 2*x1 + x2  # linear combo => 2D data!
X = np.column_stack([x1, x2, x3])

# Center
X_centered = X - X.mean(axis=0)

# METHOD 1: Eigendecomposition
cov_matrix = np.cov(X_centered.T)  # 3x3
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)
idx = np.argsort(eigenvalues)[::-1]  # sort desc
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]

# METHOD 2: SVD
U, S, Vt = np.linalg.svd(X_centered)
eigenvalues_svd = S**2 / (N - 1)
eigenvectors_svd = Vt.T
# Check: np.allclose(eigenvalues, eigenvalues_svd)

# Variance explained
var_explained = eigenvalues / eigenvalues.sum() * 100
cumulative = np.cumsum(var_explained)
# 3rd eigenvalue ~ 0 because x3 = 2*x1 + x2

# Project (reduce to 2D)
W = eigenvectors[:, :2]     # first 2 eigenvectors
X_pca = X_centered @ W      # N x 2

# Scatter plots: original vs PCA
pairs = [(0,1), (0,2), (1,2)]
fig, axes = plt.subplots(2, 3, figsize=(15,10))
for ax, (i,j) in zip(axes[0], pairs):
    ax.scatter(X[:,i], X[:,j], alpha=0.3, s=5)
X_full = X_centered @ eigenvectors
for ax, (i,j) in zip(axes[1], pairs):
    ax.scatter(X_full[:,i], X_full[:,j], alpha=0.3, s=5)
```

Key: `np.cov` takes features as rows ⇒ pass `X_centered.T`. SVD: eigenvalues $= \sigma_i^2/(N-1)$, eigenvectors = columns of V^T transposed.

3.4 Ex.2 – Rutherford Scattering

KEY FORMULAS

$\tan(\theta/2) = kZe^2/(Eb) \quad \Rightarrow \quad$ bounce-back if $b < b_{\text{crit}} = kZe^2/E$
$k = 1/(4\pi\epsilon_0), \quad E = 7.7$ MeV, $\quad Z = 79, \quad \sigma = a_0/100$

```
from scipy import constants
Z = 79; E = 7.7e6 * constants.eV
a0 = constants.physical_constants['Bohr radius'][0]
sigma = a0 / 100
epsilon_0 = constants.epsilon_0; e = constants.e
k = 1 / (4*np.pi*epsilon_0)
b_critical = k * Z * e**2 / E

N = 1_000_000
x = np.random.normal(0, sigma, N)
```

```
y = np.random.normal(0, sigma, N)
b = np.sqrt(x**2 + y**2)

n_bounced = np.sum(b < b_critical)
fraction = n_bounced / N
# Result: ~0.02% bounce back
```

3.5 Bash 2 – Data File Processing

WARNING: BASH Ex 2 PREREQUISITES

- **Data Required:** Ensure `data.csv` is already downloaded and placed in the exact same folder before running!
- **Execution Argument:** The script requires the variable n (number of copies) as an input parameter. You **MUST** run it with a number at the end:
  `./script.sh 3` *(This will create 3 copies)*

```
# 2.a: Remove comments + commas
grep -v "^#" data.csv | sed 's/,/ /g' > data.txt

# 2.b: Count even numbers
grep -oE '[0-9]+' data.txt \
    | awk '$1 % 2 == 0' | wc -l

# 2.c: Distance threshold (86.6 = 100*sqrt(3)/2)
awk '{d=sqrt($1*$1+$2*$2+$3*$3)} \
    d>86.6{a++} d<=86.6{b++} \
    END{print a, b}' data.txt

# 2.d: n scaled copies (n = $1 = script argument)
n=$1
for i in $(seq 1 $n); do
    awk -v d=$i '{print $1/d,$2/d,$3/d}' \
        data.txt > data_$i.txt
done
```

4 Version 3

4.1 Ex.6 – Route 66 Distance Grid (Broadcasting)

```
cities = ['Chicago','Springfield','Saint-Louis',
    'Tulsa','OKC','Amarillo','Santa Fe',
    'Albuquerque','Flagstaff','Los Angeles']
miles = np.array([0,198,303,736,871,
                  1175,1475,1544,1913,2448])

# Broadcasting: (10,1) - (1,10) => (10,10)
dist_mi = np.abs(miles[:,np.newaxis]
                 - miles[np.newaxis,:])
dist_km = dist_mi * 1.60934
```

Key: `[:,np.newaxis]` = column, `[np.newaxis,:]` = row. Broadcasting computes all pairwise differences.

4.2 Pandas – TDC Timing Data Analysis

TIME FORMULA — MEMORIZE!

$$t_{\text{ns}} = (\text{ORBIT} \times 3564 + \text{BX}) \times 25 + \text{TDC} \times \frac{25}{30}$$

3564 BX/orbit, each BX = 25 ns, TDC tick = 25/30≈0.833 ns

```
df = pd.read_csv('data_000637.txt', nrows=50000)

# 2. Number of BX per orbit
x = df.groupby('ORBIT_CNT')['BX_COUNTER'].max()+1
# x should be 3564

# 3. Duration
total_orbits = df['ORBIT_CNT'].max() \
             - df['ORBIT_CNT'].min()
duration_s = total_orbits * x * 25 * 1e-9

# 4. Absolute time column
df['TIME_NS'] = (df['ORBIT_CNT']*3564
    + df['BX_COUNTER'])*25 \
    + df['TDC_MEAS']*25/30

# 5. Random HEAD column
df['HEAD'] = np.random.randint(0, 2, len(df))

# 6. Filter rows
df_head1 = df[df['HEAD'] == 1].copy()

# 7. Occupancy plots (hist per FPGA)
for fpga in [0, 1]:
    data = df[df['FPGA'] == fpga]
    plt.hist(data['TDC_CHANNEL'],
             bins=range(0,140))

# 8. Top 3 noisy channels
ch = df.groupby(['FPGA','TDC_CHANNEL']).size()\
       .reset_index(name='counts')
top3 = ch.nlargest(3, 'counts')

# 9. Unique orbits
n_orbits = df['ORBIT_CNT'].nunique()
orbits_139 = df[df['TDC_CHANNEL']==139]\
    ['ORBIT_CNT'].nunique()
```

PANDAS CHEAT SHEET

`pd.read_csv(file, nrows=N)` = load data
`df.head()` = first 5 rows
`df['col']` = access column
`df[df['col']==val]` = filter rows
`.copy()` = independent copy (avoid SettingWithCopy)
`df.groupby('col')['col2'].max()` = groupby + agg
`.size()` = count per group
`.reset_index(name='x')` = group→dataframe
`.nlargest(3, 'col')` = top 3 values
`.nunique()` = count distinct values
`df['new'] = formula` = create new column

4.3 Ex.3 – Monte Carlo: Hit/Miss vs Mean Value

KEY FORMULAS

Hit/Miss: $I \approx A_{\text{rect}} \cdot k/N, \quad \sigma = A_{\text{rect}}\sqrt{k(N-k)/N^3}$
Mean value: $I \approx (b-a)\langle f \rangle, \quad \sigma = (b-a)\,\hat{\sigma}_f/\sqrt{N}$

```
def f(x):
    return np.sin(1/(x*(2-x)))**2

a, b = 0.001, 1.999  # avoid singularity!
N = 100000

# === Hit/Miss ===
x_test = np.linspace(a, b, 10000)
f_max = f(x_test).max()
x_rand = np.random.uniform(a, b, N)
y_rand = np.random.uniform(0, f_max, N)
hits = np.sum(y_rand < f(x_rand))
area_rect = (b-a) * f_max

I_hm = area_rect * hits / N
err_hm = area_rect * np.sqrt(
    hits*(N-hits)/N**3)  # binomial

# === Mean Value ===
x_rand = np.random.uniform(a, b, N)
fvals = f(x_rand)
I_mv = (b-a) * np.mean(fvals)
err_mv = (b-a) * np.std(fvals) / np.sqrt(N)  # CLT
# Mean value error << Hit/miss error
```

Why? Hit/miss: each point is hit/miss (binomial). Mean value: each point carries $f(x)$ (CLT on continuous values ⇒ smaller error).

4.4 Bash 1

Same as Version 1 (see Section 2.5).

5 Version 4

5.1 Ex.2 – Outer Product (3 Methods)

```
u = np.array([1, 3, 5, 7])
v = np.array([2, 4, 6, 8])

# Method 1: built-in
outer1 = np.outer(u, v)

# Method 2: list comprehension
outer2 = np.array(
    [[ui*vj for vj in v] for ui in u])

# Method 3: broadcasting
outer3 = u[:, np.newaxis] * v[np.newaxis, :]
```

5.2 Ex.4 – MC in High Dimensions

KEY FORMULA

$V_d = 2^d \cdot N_{\text{inside}}/N, \quad \text{exact: } V_d = \pi^{d/2}/\Gamma(d/2+1)$

```
N = 1_000_000

# === 2D circle ===
x = np.random.uniform(-1, 1, (N, 2))
r2 = np.sum(x**2, axis=1)
inside = np.sum(r2 <= 1)
area = 4 * inside / N    # 2^2 = 4

# === 10D sphere ===
d = 10
x = np.random.uniform(-1, 1, (N, d))
r2 = np.sum(x**2, axis=1)
inside = np.sum(r2 <= 1)
vol = (2**d) * inside / N  # 2^10 = 1024

# Exact: pi^(d/2) / gamma(d/2 + 1)
from scipy.special import gamma
exact = np.pi**(d/2) / gamma(d/2 + 1)
```

Recipe: Sample in $[-1,1]^d$ → count $\sum x_i^2 \leq 1$ → multiply by 2^d.

5.3 Ex.5 – FFT Image Filtering

RECIPE

1. `fft2` → `fftshift` → log-scale `imshow`
2. Create circular mask centered at $(r/2, c/2)$
3. Apply mask to shifted FFT
4. `ifftshift` → `ifft2` → `np.real`

```
from scipy import fftpack

img = plt.imread('moonlanding.png')
if len(img.shape)==3: img = img.mean(axis=2)

# Forward FFT
F = fftpack.fft2(img)
F_shifted = fftpack.fftshift(F)

# Plot spectrum (LOG SCALE!)
plt.imshow(np.log(np.abs(F_shifted)+1), cmap='gray')

# Low-pass mask (circle of radius r=60)
rows, cols = img.shape
crow, ccol = rows//2, cols//2
mask = np.zeros((rows, cols))
y, x = np.ogrid[:rows, :cols]
mask[(x-ccol)**2 + (y-crow)**2 <= 60**2] = 1

# Apply filter + inverse FFT
F_filtered = F_shifted * mask
F_ishifted = fftpack.ifftshift(F_filtered)
img_clean = np.real(fftpack.ifft2(F_ishifted))

plt.imshow(img_clean, cmap='gray')
```

Key: `np.ogrid` creates index arrays for mask. Log scale needed because FFT magnitudes span many orders.

5.4 Ex.3 – Profile Plot + Linear Regression

```
data = np.load('residuals_261.npy',
               allow_pickle=True).item()
df = pd.DataFrame(data)

# Clean: |residual| < 2
df_clean = df[np.abs(df['residuals']) < 2].copy()

# Linear regression
slope, intercept, r, p, se = stats.linregress(
    df_clean['distances'], df_clean['residuals'])

# Profile plot: bin distance, compute mean+std
n_bins = 20
bin_edges = np.linspace(0, 20, n_bins+1)
bin_centers = (bin_edges[:-1]+bin_edges[1:])/2
y = np.zeros(n_bins); erry = np.zeros(n_bins)

for i in range(n_bins):
    mask = (df_clean['distances'] >= bin_edges[i])\
         & (df_clean['distances'] < bin_edges[i+1])
    if mask.sum() > 0:
        y[i] = df_clean.loc[mask,'residuals'].mean()
        erry[i] = df_clean.loc[mask,'residuals'].std()

plt.scatter(df_clean['distances'],
            df_clean['residuals'], alpha=0.3, s=5)
plt.errorbar(bin_centers, y, yerr=erry,
             fmt='ro-', capsize=3)
```

Key: Load `.npy` with `allow_pickle=True` then `.item()` then `pd.DataFrame()`.

5.5 Bash 2

Same as Version 2 (see Section 3.5).

6 Version 5

6.1 Ex.8 – Random Walk Diffusion

KEY RESULT

$\langle X^2(t)\rangle = t \quad \Rightarrow \quad \text{RMS} = \sqrt{t}$

```
n_walkers = 1000; n_steps = 200

steps = np.random.choice([-1,1],
    size=(n_walkers, n_steps))
positions = np.cumsum(steps, axis=1)
mean_sq = np.mean(positions**2, axis=0)
rms = np.sqrt(mean_sq)

time = np.arange(1, n_steps+1)
plt.plot(time, rms, 'b-', label='Simulated')
plt.plot(time, np.sqrt(time), 'r--', label='sqrt(t)')
```

Key: `cumsum(axis=1)` = cumulative sum along columns (time). `mean(axis=0)` = average over walkers (rows).

6.2 Ex.2 – Color-coded Scatter Plot

```
def generate_2d_categories(n_pts=200,
                           n_cat=2, seed=42):
    np.random.seed(seed)
    means = [(i*1.5, (i%2)*1.5)
             for i in range(n_cat)]
    stds = [0.3] * n_cat
    data, labels = [], []
    for i, (m, s) in enumerate(zip(means, stds)):
        x = np.random.normal(m[0], s, n_pts)
        y = np.random.normal(m[1], s, n_pts)
        data.append(np.column_stack([x, y]))
        labels.append(np.full(n_pts, i))
    return np.vstack(data), np.hstack(labels)

data, labels = generate_2d_categories(200, 2)
for i in range(2):
    mask = labels == i
    plt.scatter(data[mask,0], data[mask,1],
                label=f'Cat {i+1}')
# For n categories: use cmap = plt.cm.get_cmap('tab10')
```

6.3 Ex.1 – Wind Speed Prediction (50-year)

RECIPE

1. $p_i = i/(N+1)$ (cumulative probability)
2. Sort speeds, pair with p_i
3. `UnivariateSpline(cprob, sorted_speeds, s=0)`
4. 50-year = $Q(0.98)$ (upper 2%)

```
from scipy.interpolate import UnivariateSpline

max_speeds = np.load('max-speeds.npy')
years_nb = max_speeds.shape[0]  # 21

cprob = np.arange(1, years_nb+1) / (years_nb+1)
sorted_speeds = np.sort(max_speeds)

quantile_func = UnivariateSpline(
    cprob, sorted_speeds, s=0)
```

```
fifty_prob = 1 - 0.02  # = 0.98
fifty_wind = quantile_func(fifty_prob)

# Plot
p_range = np.linspace(0.01, 0.99, 100)
plt.plot(cprob, sorted_speeds, 'bo')
plt.plot(p_range, quantile_func(p_range), 'r-')
```

6.4 Ex.5 – MC with Importance Sampling

KEY FORMULAS

$$I = \int_0^1 \frac{x^{-1/2}}{e^x + 1}\, dx \qquad w(x) = 1/\sqrt{x}$$

PDF: $p(x) = 1/(2\sqrt{x}) \quad \Rightarrow \quad$ sample $x = u^2$

$$I = \underbrace{\int_0^1 w(x)\, dx}_{=2} \cdot \langle g(x) \rangle_p = 2 \cdot \overline{g} \quad \text{where } g(x) = 1/(e^x{+}1)$$

```
N = 1_000_000
u = np.random.uniform(0, 1, N)
x = u**2  # samples from p(x) = 1/(2*sqrt(x))

g = 1 / (np.exp(x) + 1)  # f(x)/w(x)

integral = 2 * np.mean(g)  # ~0.84
error = 2 * np.std(g) / np.sqrt(N)
```

Why $x = u^2$? CDF of $p(x) = 1/(2\sqrt{x})$ is $P(x) = \sqrt{x}$. Inverse: $x = P^{-1}(u) = u^2$.

6.5 Bash 1

Same as Version 1 (see Section 2.5).

7 Quick Reference Tables

NUMPY TRICKS TO REMEMBER

`data.T` = transpose (unpack columns)
`np.column_stack([a,b,c])` = stack as columns
`np.argmax(A, axis=0)` = index of max per column
`np.corrcoef(A)` = correlation matrix
`np.cumsum(A, axis=1)` = cumsum along cols
`np.argsort(x)[::-1]` = indices for descending sort
`A[:,np.newaxis]` = add new axis (broadcasting)
`np.meshgrid(x,y)` = 2D grid from 1D arrays
`np.ogrid[:r,:c]` = open grid (for masks)
`np.allclose(a,b)` = check arrays nearly equal
`np.random.choice([-1,1], (m,n))` = random ± 1

SCIPY FUNCTIONS TO REMEMBER

`curve_fit(func, x, y, p0=[...])` → popt, pcov
`minimize(func, x0, method='BFGS')` → res.x, res.fun
`stats.linregress(x,y)` → slope, intercept, r, p, se
`stats.norm(loc, scale).pdf(x)` = Gaussian PDF
`integrate.trapezoid(y, x)` = trapezoidal integral
`fftpack.fft2(img)` / `ifft2(F)` = 2D FFT
`fftpack.fftshift(F)` / `ifftshift(F)`
`UnivariateSpline(x, y, s=0)` = exact spline
`gamma(n)` = $\Gamma(n)$ function
`constants.eV`, `constants.e`, `constants.epsilon_0`

MATPLOTLIB PATTERNS

`fig, (ax1,ax2) = plt.subplots(1,2)`
`fig, axes = plt.subplots(2,3)` then `axes[row]`
`plt.imshow(Z, extent=[...], origin='lower')`
`plt.errorbar(x,y,yerr=e, fmt='ro-', capsize=3)`
`plt.scatter(x,y, c=colors, s=size, alpha=0.3)`
`ax.yaxis.set_major_locator(MaxNLocator(integer=True))`
`plt.colorbar(label='...')` after imshow

EXAM STRUCTURE: 3-5 Python exercises + 1 Bash exercise.
2 Bash variants (students / data processing).
Always set `np.random.seed(42)` for reproducibility

BASH WORKFLOW: HOW TO TEST AT THE EXAM

1. **Create/Edit list:** `nano script.sh`
2. **Shebang:** Always write `#!/bin/bash` at the very first line!
3. **Save & Exit:** Press `Ctrl+O` → `Enter` → `Ctrl+X`
4. **Make Executable:** `chmod +x script.sh`
5. **Execute:** `./script.sh`
6. **Verify Output:** `ls -l` (check created files) or `cat file.csv` (read output)

8 One-Page Formula Sheet

All Formulas Needed

Radioactive Decay (V1)

$$p_{\text{decay}} = 1 - 2^{-\Delta t/\tau} \tag{1}$$

$$\text{PDF: } p(t) = \frac{\ln 2}{\tau}\, 2^{-t/\tau} \tag{2}$$

$$\text{Inverse CDF: } t = -\tau\, \log_2(r), \quad r \sim \mathcal{U}(0,1) \tag{3}$$

Sinusoidal Fit (V1)

$$T(x) = A \sin\left(\frac{2\pi(x-C)}{12}\right) + D$$

KDE Bandwidth (V1)

$$h = 1.06\, \hat{\sigma}\, N^{-1/5} \quad \text{(Scott's rule)}$$

$$\text{KDE}(x) = \frac{1}{N}\sum_{i=1}^{N} \frac{1}{h\sqrt{2\pi}} \exp\left(-\frac{(x-x_i)^2}{2h^2}\right)$$

PCA (V2)

$$\mathbf{C} = \text{cov}(\mathbf{X}_c^T), \quad \mathbf{C}\mathbf{v}_i = \lambda_i \mathbf{v}_i$$

$$\text{SVD: } \lambda_i = \frac{\sigma_i^2}{N-1}, \quad \text{Var. explained}_i = \frac{\lambda_i}{\sum_j \lambda_j}$$

$$\mathbf{X}_{\text{proj}} = \mathbf{X}_c \cdot \mathbf{W} \quad (\mathbf{W} = \text{top-}k \text{ eigenvectors})$$

Camelback (V2)

$$f(x,y) = \left(4 - 2.1x^2 + \tfrac{x^4}{3}\right)x^2 + xy + (4y^2 - 4)y^2$$

$$\text{Global min: } f(\pm 0.0898, \mp 0.7126) \approx -1.0316$$

Rutherford Scattering (V2)

$$b_{\text{crit}} = \frac{kZe^2}{E}, \quad k = \frac{1}{4\pi\epsilon_0}$$

$$\text{Fraction bounced} = P(b < b_{\text{crit}}), \quad b = \sqrt{x^2+y^2}$$

MC Hit/Miss (V3)

$$I = A_{\text{rect}} \cdot \frac{k}{N}, \qquad \sigma = A_{\text{rect}} \sqrt{\frac{k(N-k)}{N^3}}$$

MC Mean Value (V3)

$$I = (b-a)\,\overline{f}, \qquad \sigma = (b-a)\,\frac{\hat{\sigma}_f}{\sqrt{N}}$$

TDC Timing (V3)

$$\boxed{t_{\text{ns}} = (\text{ORBIT} \times 3564 + \text{BX}) \times 25 + \text{TDC} \times \frac{25}{30}}$$

Volume of d-Sphere (V4)

$$V_d^{\text{MC}} = 2^d \cdot \frac{N_{\text{inside}}}{N}, \qquad V_d^{\text{exact}} = \frac{\pi^{d/2}}{\Gamma(d/2+1)}$$

FFT Filtering Pipeline (V4)

$$\text{img} \xrightarrow{\text{fft2}} F \xrightarrow{\text{fftshift}} F_s \xrightarrow{\times \text{mask}} F_f \xrightarrow{\text{ifftshift}} F' \xrightarrow{\text{ifft2}} \text{img\_clean}$$

$$\text{Low-pass mask: } (x-c_x)^2 + (y-c_y)^2 \le R^2$$

Random Walk (V5)

$$\langle X^2(t) \rangle = t \quad \Rightarrow \quad \text{RMS} = \sqrt{t}$$

Wind Speed Quantile (V5)

$$p_i = \frac{i}{N+1}, \qquad Q(0.98) = \text{50-year wind speed (upper 2\%)}$$

Importance Sampling (V5)

$$I = \int_0^1 \frac{x^{-1/2}}{e^x+1}\,dx, \quad w(x) = x^{-1/2}, \quad p(x) = \frac{1}{2\sqrt{x}}$$

$$\text{Sample: } x = u^2\ (u \sim \mathcal{U}), \quad I = 2\,\overline{g}, \quad g(x) = \frac{1}{e^x+1}$$

Broadcasting Rule

$$\mathbf{u}_{(n\times 1)} \circ \mathbf{v}_{(1\times m)} \to \mathbf{M}_{(n\times m)} \quad \text{(outer product / distance grid)}$$

Error Formulas Summary

Method	Estimate	Error
Hit/Miss	$A \cdot k/N$	$A\sqrt{k(N-k)/N^3}$
Mean Value	$(b-a)\bar{f}$	$(b-a)\hat{\sigma}_f/\sqrt{N}$
Importance	$C \cdot \bar{g}$	$C \cdot \hat{\sigma}_g/\sqrt{N}$

9 Pseudo-Code Summaries

Structure & Steps

V1 – Ex.9: Populations

1. Load text file → unpack 4 columns (year, hares, lynxes, carrots)
2. Plot 3 time series on same figure with legend
3. Compute mean and std for each species
4. Compute 3×3 correlation matrix
5. Stack 3 species into matrix $(3 \times N_{\text{years}})$
6. For each year: find index of max across 3 species → print name

V1 – Ex.2: Alaska Temperature

1. Define data: months 1–12, max temps array, min temps array
2. Define sinusoidal model: $f(x) = A\sin(2\pi(x - C)/12) + D$
3. Set initial guesses (amplitude, phase, offset) for max and min
4. Fit model to max temps → get optimal parameters
5. Fit model to min temps → get optimal parameters
6. Plot data points + smooth fitted curves on same figure

V1 – Ex.1: Radioactive Decay

M1: *Step-by-step simulation:*

(a) Set N_0, τ, Δt. Compute $p = 1 - 2^{-\Delta t/\tau}$
(b) Loop over time steps: for each surviving atom, draw random $< p$ → it decays
(c) Track Tl count (decreasing) and Pb count (increasing)
(d) Plot both + theoretical $N_0 \cdot 2^{-t/\tau}$

M2: *Inverse transform:*

(a) Draw N_0 uniform randoms $r \in (0, 1)$
(b) Convert: $t_{\text{decay}} = -\tau \log_2(r)$
(c) Sort decay times. For each plot-time, count how many have decayed
(d) Compare with theoretical curve

V1 – Ex.1: KDE

1. Generate N samples from Gaussian(5, 2)
2. Histogram: get counts, bin edges, centers, width
3. Compute Poisson errors $= \sqrt{\text{counts}}$, plot error bars
4. Compute bandwidth: $h = 1.06 \cdot \text{std} \cdot N^{-1/5}$
5. For each data point: create Gaussian PDF centered there
6. Sum all Gaussians over fine x-grid
7. Normalize KDE to match histogram area (scale by ratio of integrals)
8. Plot KDE curve over histogram

V1 & V3 & V5 – Bash 1: Students

1. Create directory, cd into it, download CSV if not exists
2. grep "PoD" → pod.csv, grep "Physics" → physics.csv
3. Loop A–Z: count lines starting with each letter
4. Find letter with max count using if/comparison
5. Split into 18 groups using awk modulo on line number

V2 – Ex.3: Camelback

1. Define $f(x, y)$ (six-hump camelback function)
2. Create 2D meshgrid, evaluate f on grid, display with imshow
3. Choose 5 starting points spread across domain
4. For each: run BFGS minimizer → store (position, value)
5. Compare all results: smallest value = global minimum
6. Note: starting at $(0, 0)$ finds local min, not global

V2 – Ex.4: Linspace & Slicing

1. Create array 0 to 2π with 100 points
2. Slice every 10th element, reverse array
3. Compute $|\sin(x) - \cos(x)|$, find where < 0.1 (boolean mask)
4. Plot sin and cos, highlight intersection points

V2 – Ex.1: PCA

1. Generate 3 features: $x_1 \sim \mathcal{N}$, $x_2 = x_1 + \text{noise}$, $x_3 = 2x_1 + x_2$
2. Stack as $(N \times 3)$ matrix, center (subtract column means)

M1: Compute 3×3 covariance matrix → eigendecompose → sort descending

M2: SVD of centered matrix → eigenvalues $= \sigma^2/(N-1)$

3. Compute variance explained per component (3rd ≈ 0)
4. Project: multiply centered data by top-2 eigenvectors
5. Plot: 3 scatter pairs original vs 3 scatter pairs in PCA space

V2 – Ex.2: Rutherford

1. Set constants: $Z = 79$, $E = 7.7$ MeV (convert to Joules), Bohr radius, $\sigma = a_0/100$
2. Compute $b_{\text{crit}} = kZe^2/E$
3. Generate N particles: $x, y \sim \mathcal{N}(0, \sigma)$, compute $b = \sqrt{x^2 + y^2}$
4. Count $b < b_{\text{crit}}$ → fraction bounced back

V2 & V4 – Bash 2: Data Processing

1. Remove comment lines (grep -v "^#"), replace commas with spaces (sed)
2. Count even numbers: extract all numbers, filter even, count
3. Compute 3D distance per row, count above/below $100\sqrt{3}/2$
4. Loop $i = 1..n$: divide all columns by i, save to new file

V3 – Ex.6: Route 66 Distance Grid

1. Define city names and cumulative mile distances (10 cities)
2. Reshape miles: column vector − row vector → 10×10 matrix (broadcasting)
3. Take absolute value → pairwise distance matrix
4. Convert miles to km ($\times 1.60934$)

V3 – Pandas: TDC Timing

1. Load CSV (first 50k rows) into DataFrame
2. GroupBy ORBIT → max BX per orbit +1 (should be 3564)
3. Duration: (max−min orbit) $\times$ 3564 $\times$ 25 ns → seconds
4. New column: $t = (\text{ORBIT} \times 3564 + \text{BX}) \times 25 + \text{TDC} \times$

25/30
5. New column HEAD: random 0/1
6. Filter: keep only HEAD=1 rows (use `.copy()`)
7. Occupancy: histogram of TDC_CHANNEL per FPGA
8. GroupBy (FPGA, CHANNEL) → size → nlargest(3) = top noisy
9. Count unique orbits total, and unique orbits for channel 139

V3 – Ex.3: MC Hit/Miss vs Mean Value

1. Define $f(x) = \sin^2\left(\frac{1}{x(2-x)}\right)$, domain $(0.001, 1.999)$

H/M: Find $f_{\max}$, draw (x, y) uniform in rectangle, count $y < f(x)$ = hits
Integral = $A_{\text{rect}} \cdot k/N$, error = binomial formula

MV: Draw x uniform, evaluate $f(x)$, integral = $(b-a) \cdot \text{mean}(f)$
Error = $(b-a) \cdot \text{std}(f)/\sqrt{N}$ (CLT)

2. Compare: Mean Value has much smaller error

V4 – Ex.2: Outer Product

1. Define vectors **u** and **v**
2. Method 1: `np.outer(u, v)` (built-in)
3. Method 2: nested list comprehension
4. Method 3: broadcasting $\mathbf{u}_{(:,\text{newaxis})} \times \mathbf{v}_{(\text{newaxis},:)}$

V4 – Ex.4: MC High Dimensions

1. Draw N points uniform in $[-1, 1]^d$
2. Compute $r^2 = \sum x_i^2$ for each point
3. Count inside: $r^2 \leq 1$
4. Volume = $2^d \cdot N_{\text{inside}}/N$
5. Compare with exact: $\pi^{d/2}/\Gamma(d/2+1)$
6. Run for $d = 2, 3, 5, 10$ (efficiency drops in high d)

V4 – Ex.5: FFT Image Filtering

1. Load image, convert to grayscale if needed
2. Forward: fft2 → fftshift (center low frequencies)
3. Display: log of magnitude (log scale essential!)
4. Create circular mask: use ogrid, radius threshold
5. Multiply shifted FFT by mask (keep low frequencies)
6. Inverse: ifftshift → ifft2 → take real part
7. Display cleaned image

V4 – Ex.3: Profile Plot

1. Load .npy dict → convert to DataFrame
2. Clean: keep |residual| < 2
3. Linear regression on cleaned data → slope, intercept, r
4. Profile: bin distances into 20 bins
5. For each bin: compute mean and std of residuals
6. Plot: scatter (all points) + error bars (bin means ± std)

V5 – Ex.8: Random Walk

1. Generate $(W \times S)$ matrix of ± 1 random steps
2. Cumulative sum along time axis → positions
3. Square positions, average over walkers → $\langle X^2(t) \rangle$
4. RMS = $\sqrt{\langle X^2 \rangle}$, plot vs $\sqrt{t}$ (should match)

V5 – Ex.2: Scatter Plot

1. Generate 2D clustered data: n categories, each with own mean
2. For each category: draw (x, y) from Gaussian around that mean
3. Stack all data + labels
4. Plot: separate scatter per category with different color

V5 – Ex.1: Wind Speed

1. Load max speed data (N years)
2. Assign cumulative probabilities: $p_i = i/(N+1)$
3. Sort speeds ascending, pair with p_i
4. Fit spline through (p, speed) pairs (exact: $s = 0$)
5. 50-year return: evaluate spline at $p = 0.98$
6. Plot data points + spline curve

V5 – Ex.5: Importance Sampling

1. Identify weight function $w(x) = 1/\sqrt{x}$, so $g(x) = f/w = 1/(e^x + 1)$
2. PDF: $p(x) = w(x)/\int w = 1/(2\sqrt{x})$
3. CDF inversion: $x = u^2$ where $u \sim \mathcal{U}(0, 1)$
4. Draw N samples via u^2, evaluate g at each
5. Integral = $\left(\int w\right) \cdot \text{mean}(g) = 2\,\overline{g}$
6. Error = $2 \cdot \text{std}(g)/\sqrt{N}$